LBRO

Interview & Architecture
Master Booklet

Engineering / Interview Preparation Document
Repository-First Analysis

Source: Actual LBRO codebase implementation
Not a marketing document

Table of Contents

LBRO Interview & Architecture Master Booklet 2
1. Executive Summary - What Is LBRO? 2
2. System Architecture 2
3. Technology Stack - Why Each Choice 2
4. Request Lifecycle - Security Event Flow 3
5. User Workflow 3
6. Authentication Deep Dive 3
7. RBAC & Multi-Tenancy (Critical Section) 4
8. Database Deep Dive 5
9. API Design - Key Endpoints 5
10. Event Ingestion & SDK 6
11. ML Pipeline 6
12. Evidence Vault 7
13. Compliance Engine 7
14. Evidence + Compliance + ML Relationship 8
15. Security Architecture 8
16. Security Incidents & Remediation History 8
17. DevOps & Docker 9
18. AWS & Terraform 9
19. CI/CD 9
20. Engineering Trade-Offs 9
21. Challenges Faced (From Audit History) 10
22. Testing Strategy 10
23. Performance & Scalability 10
24. Failure Scenarios 10
25. Presentation Guide (10-15 min) 10
26. Live Demo Script 11
27. Interview Questions & Answers (50+) 11
28. Hard Interview Questions - Design Challenges 15
29. Elevator Pitches 15
30. Personal Contribution 16
31. Glossary 16
32. Final Cheat Sheet 16
33. Source Reference Index 17
34. Appendix A - Complete Migration History 18
35. Appendix B - Frontend Pages & Routes 18
36. Appendix C - Sample Event Ingest Request 19
37. Appendix D - RBAC Permission Matrix (Summary) 19
38. Appendix E - Documentation vs Implementation Conflicts (Complete List) 19
39. Appendix F - Red-Team Test Scenarios 20

LBRO Interview & Architecture Master Booklet

1. Executive Summary - What Is LBRO?

LBRO (Legal Breach Response Operations) is a **multi-tenant security incident response platform** built as a full-stack web application. It ingests security events from external applications via project-scoped API keys, classifies them using ML (GaussianNB on CICIDS2017-style features) with a heuristic fallback, auto-creates incidents for high-severity events, preserves evidence with SHA-256 integrity and chain-of-custody, tracks regulatory compliance obligations (GDPR, HIPAA,

DPDPA), and provides role-based dashboards for analysts and administrators.

Problem: Security teams need a unified workflow from raw security telemetry -> classified incident -> evidence preservation -> regulatory notification deadlines - not just a log viewer.

Solution: LBRO connects external apps via SDK/API keys, processes events synchronously in the API (with async SQS workers for incident re-classification and notification dispatch), enforces strict project isolation, and surfaces compliance deadlines derived from real incident metadata.

Users: Security analysts, incident responders, compliance officers, project administrators, platform super-admins.

Source: `README.md`, `backend/app/main.py`, `backend/app/routers/events.py`

2. System Architecture

2.1 High-Level Diagram

```
                    +-------------------------------------------------------+
```

2.2 Component Responsibilities

Component	Purpose	Key Files
Frontend	SPA dashboard, incident/evidence/	`frontend/src/`
Backend API	REST API, auth, RBAC, event inges	`backend/app/main.py`, `routers/`
PostgreSQL	All persistent data including evi	`backend/app/models/`, migrations
Worker	SQS poll: incident ML + notificat	`backend/app/workers/main.py`
ML Classifier	GaussianNB + heuristic fallback	`backend/app/ml/classifier.py`
Terraform	AWS IaC template (ECS/RDS/SQS/S3)	`terraform/main.tf`
Docker	Local/prod container orchestratio	`docker-compose.yml`, `docker-com

Source: `docker-compose.yml`, `backend/app/workers/main.py`, `terraform/main.tf`

3. Technology Stack - Why Each Choice

Technology	Purpose	Why Chosen	Alternative	Trade-off
Python 3.11+	Backend runtime	ML ecosystem, FastAP	Node.js, Go	GIL limits CPU-bound
FastAPI	HTTP API	Async, Pydantic vali	Flask, Django	Less batteries-inclu
React 18 + TS	Frontend SPA	Component model, typ	Vue, Svelte	Build complexity
PostgreSQL 16	Primary DB	ACID, JSON, BYTEA fo	MongoDB	BYTEA evidence is ex
SQLAlchemy 2 + Alemb	ORM + migrations	Type-safe models, ve	raw SQL	ORM learning curve
scikit-learn	ML inference	Tabular RF/NB, pickl	PyTorch	Not for unstructured
Docker	Packaging	Reproducible dev/pro	bare metal	Image size overhead
AWS SQS	Async job queue	Managed, DLQ support	Kafka, Redis	Not used for raw eve
Terraform	IaC	Declarative AWS prov	CloudFormation	State management com
pytest / Vitest	Testing	Backend/frontend uni	unittest	-
bcrypt (Passlib)	Password hash	Industry standard	argon2	-
HMAC-SHA256	API key hash	Fast lookup by prefi	bcrypt per key	Requires SECRET_KEY

JWT HS256	Session tokens	Stateless auth, refr	server sessions	Revocation needs JTI

Source: `backend/requirements.txt`, `frontend/package.json`, `backend/app/core/security.py`, `backend/app/core/api_keys.py`

4. Request Lifecycle - Security Event Flow

4.1 End-to-End Pipeline

```
External Application
                                                                    -> SDK
```

4.2 Stage Details

Authentication: `backend/app/dependencies.py::get_project_from_api_key()` - prefix lookup on `projects.api_key_prefix`, HMAC verify via `backend/app/core/api_keys.py::verify_api_key()`.

Security contract: `backend/app/routers/events.py` lines 6-8 - project_id from API key only.

ML classification: `_classify_event()` calls `LBROClassifier.predict()` if ?10 non-zero features; else severity->attack mapping table.

Failure behavior: Invalid API key -> 401; invalid event_type -> 422; DB error -> 500 with rollback.

Source: `backend/app/routers/events.py`, `backend/app/ml/classifier.py`

5. User Workflow

```
Register (email/password) -> JWT access + refresh tokens
                                                                    -> Onb
```

Multi-project: Users can belong to multiple projects via `project_members`. `accessible_project_ids()` in `backend/app/core/project_access.py` returns union of owned + member projects.

Super Admin: Platform role `super_admin` bypasses project restrictions (audit-logged). Routes under `/api/v1/platform/`.

Source: `frontend/src/routes/AppRouter.tsx`, `backend/app/core/project_access.py`, `backend/app/services/invitation_service.py`, `docs/MULTI_PROJECT.md`

6. Authentication Deep Dive

6.1 Registration & Login

- **Register:** `POST /api/v1/auth/register` - email, username, password. Role defaults to `viewer` (P0 fix: was `admin`). Gated by `ALLOW_PUBLIC_REGISTRATION` (`True` in docker-compose, `False` recommended for prod).

- **Login:*

6.2 JWT Structure

Access token claims: `sub` (user id), `exp`, `type: "access"`, `jti`, `role`, `email`, `permissions`

Refresh token claims: `sub`, `exp`, `type: "refresh"` (no jti)

Expiry: Access 30 min (`ACCESS_TOKEN_EXPIRE_MINUTES=30`). README claims 1 hour - **MISMATCH**.

Algorithm: HS256 with `SECRET_KEY`

6.3 Logout & Revocation

`POST /api/v1/auth/logout` inserts `jti` into `revoked_tokens` table. `get_current_user()` checks blacklist before accepting token.

Source: `backend/app/routers/auth.py`, `backend/app/models/revoked_token.py`, `backend/app/services/auth_service.py`, migration `005_revoked_tokens.py`

6.4 Frontend Token Storage

Access token in module memory (XSS-resistant vs localStorage). Refresh token in sessionStorage. Axios interceptor refreshes proactively.

Source: `frontend/src/store/authStore.ts`

7. RBAC & Multi-Tenancy (Critical Section)

7.1 Two-Level Role Model

Platform level: `super_admin` - all permissions + platform permissions

Project level: `admin` > `analyst` > `viewer`

Permissions: 32 enumerated in `backend/app/core/rbac.py::Permission` - mapped via `ROLE_PERMISSIONS` dict. Never compare role strings directly; use `has_permission()`.

7.2 Authorization Decision Flow

```
  Request arrives
                                                
```

8.2 Key Tables

Table	Purpose	Security Notes
`users`	Identity, platform role, legacy u	Password bcrypt; lockout fields
`projects`	Tenant boundary	`api_key_hash` + `api_key_prefix`
`project_members`	Multi-user project access	role: admin/analyst/viewer
`project_invitations`	Pending invites	token HMAC-hashed, 7-day TTL
`security_events`	Raw ingested events	Scoped by project_id
`incidents`	Classified security incidents	project_id FK, ML metadata
`evidence`	File bytes + metadata	`file_data` BYTEA, `sha256_hash`,
`chain_of_custody`	Access audit trail	hash_at_time on each action
`compliance_records`	Per-incident regulatory obligatio	Derived from incident jurisdictio
`revoked_tokens`	JWT JTI blacklist	PK on jti

Migrations: 14 versions, head `014_project_invitations`. Source: `backend/app/migrations/versions/`

8.3 Design Decisions

Why PostgreSQL? ACID transactions for incident+evidence+compliance consistency; FK constraints enforce referential integrity; BYTEA for evidence (current impl).

Why project_membership table? Enables multi-user projects without duplicating project data; supports invitations and role granularity.

Why hash API keys? Migration `012_hash_api_keys.py` - DB dump no longer exposes plaintext keys. Lookup: prefix index -> HMAC verify loop.

Why one-time API key reveal? Key shown only at generation/rotation; only hash stored.

Source: `backend/app/models/`, `backend/app/migrations/versions/012_hash_api_keys.py`, `LBRO_Engineering_Audit.md`

9. API Design - Key Endpoints

Method	Route	Auth	Purpose
POST	`/auth/register`	Public*	User registration
POST	`/auth/login`	Public	Email/password -> JWT
POST	`/auth/refresh`	Refresh token	New access token
POST	`/auth/logout`	JWT	Revoke JTI
GET	`/auth/me`	JWT	Current user profile
POST	`/events`	Project API key	Ingest security event
POST	`/events/batch`	Project API key	Batch ingest (max 100)
GET	`/events/stream`	Project API key	SSE live stream
GET/POST	`/incidents`	JWT + project scope	List/create incidents
POST	`/incidents/{id}/evidence	JWT + permission	Upload evidence
GET	`/evidence/{id}/download`	JWT + project scope	Download file

POST	`/evidence/{id}/verify`	JWT	SHA-256 integrity check
GET	`/compliance/dashboard`	JWT + project	Compliance metrics
POST	`/compliance/obligations`	JWT + admin	Manage obligations
GET	`/projects`	JWT	List accessible projects
POST	`/projects/{id}/invitatio	JWT + admin	Invite member
POST	`/invitations/accept`	JWT	Accept invitation
GET	`/dashboard/summary`	JWT + project	Dashboard metrics
GET	`/reports/weekly`	JWT + project	Weekly report JSON/PDF
GET	`/platform/*`	super_admin	Platform administration
GET	`/health`	Public	Health check

*When `ALLOW_PUBLIC_REGISTRATION=True`

Source: `backend/app/main.py` router registration, individual routers in `backend/app/routers/`

10. Event Ingestion & SDK

10.1 External Connection

1. Create project in UI 2. Generate API key (`proj_...`) - shown once 3. Configure SDK or curl with `Authorization: Bearer proj_...` 4. POST JSON event payload to `/api/v1/events`

10.2 Security Design

Project from key, not body: Prevents attacker from sending events to another tenant's project.

Rate limiting: 60 req/min default (`RATE_LIMIT_PER_MINUTE=60`). README says 100 - **MISMATCH**.

10.3 SDK Status

SDK directory: `sdk/` contains Go and Java **stubs/examples** - minimal reference clients, not a mature published package.

Source: `sdk/`, `backend/app/routers/projects.py` (SDK download endpoint)

11. ML Pipeline

11.1 Model Details

Active version: `v2.0.0-nb-tuned` (GaussianNB) per `backend/app/ml/models/registry.json`

Features: 77 CICIDS2017 network flow features (`backend/app/ml/features.py`). README claims 78 - **MISMATCH**.

Classes: 15 attack categories including BENIGN, DDoS, PortScan, Web Attack variants.

Artifacts: `cicids2017_classifier.pkl`, `scaler.pkl` - paths configured in `config.py`. **No .pkl files committed to repo** - heuristic fallback active unless deployed with model files.

11.2 Inference Flow

```
  Features extracted from event payload
                                                                    -> If
```

11.3 Why GaussianNB / Random Forest Class?

Repository evidence: Registry specifies GaussianNB (Naive Bayes), not Random Forest. README may reference RF from earlier iteration.

Reasons for tabular classical ML on LBRO:

- Interpretable feature->class mapping for interviews

- Fast CPU

Algorithm	Pros	Cons for LBRO
Logistic Regression	Simple, fast	Linear decision boundaries
Decision Tree	Interpretable	Overfits
Random Forest	Robust ensemble	Heavier than NB
XGBoost	High accuracy	Tuning complexity
Neural Network	Flexible	Overkill for 77 tabular features;

11.4 Fallback When ML Unavailable

`LBROClassifier` returns heuristic classification with `model_version="heuristic-fallback"`, confidence 0.65, `needs_review=True`. System remains operational.

Worker re-classification: Incidents enqueued to SQS for async ML re-run via `incident_worker.py`.

Source: `backend/app/ml/classifier.py`, `backend/app/workers/incident_worker.py`, `backend/app/config.py`

12. Evidence Vault

12.1 Storage Implementation

CURRENT IMPLEMENTATION: Evidence files stored in PostgreSQL `evidence.file_data` (BYTEA, deferred load). Migration `006_evidence_postgres_storage.py`.

NOT IMPLEMENTED at runtime: S3 Object Lock / presigned URL download despite README and Terraform S3 bucket definitions.

Download: Backend serves `/api/v1/evidence/{id}/download` - `evidence_service.get_download_url()`.

12.2 Integrity & Chain of Custody

SHA-256: Computed at upload via `compute_sha256()` - stored in `sha256_hash`. Verify endpoint recomputes and compares.

Chain of custody: `ChainOfCustody` record on upload, download, and verify - captures `performed_by`, `hash_at_time`, `ip_address`, `action`.

Immutability: `is_immutable=True` default; delete blocked for immutable evidence.

Project isolation: Evidence queries JOIN through `Incident.project_id` when scoped.

Source: `backend/app/services/evidence_service.py`, `backend/app/models/evidence.py`, `backend/app/routers/evidence.py`

13. Compliance Engine

13.1 Implemented Regulations

Framework	Deadline	Trigger Conditions
GDPR	72 hours	Jurisdiction EU/EEA/UK + personal
HIPAA	60 days	US jurisdiction OR health_data_in
DPDPA	72 hours	IN jurisdiction + personal_data_i

Source: `REGULATION_RULES` in `backend/app/services/compliance_service.py`

13.2 Percentage Calculation - No Fabrication

`compliance_percentage(met, total)` in `backend/app/core/metrics.py`:

- Returns `round(met/total*100, 2)` only when `total > 0`

- Returns

13.3 Status Scenarios

- **No data:** Percentage `None` or 0 obligations shown

- **Partial:

14. Evidence + Compliance + ML Relationship

```
  Security Event (ingest)
                                                                                    -> ML/
```

This pipeline makes LBRO an **incident response workflow**, not merely a log dashboard.

Source: `backend/app/routers/events.py`, `backend/app/services/compliance_service.py`, `backend/app/services/notification_service.py`

15. Security Architecture

Threat	Control	Implementation	Test
Stolen password	bcrypt + lockout	`security.py`, `auth_serv	`test_auth.py`
Stolen JWT	Short expiry + JTI revoke	`revoked_tokens`, logout	auth tests
API key leak	HMAC hash at rest, prefix	`api_keys.py`, migration	`test_api_keys.py`
IDOR	project_id scoping on all	`project_access.py`, serv	`test_project_isolation.p
Privilege escalation	viewer default on registe	`auth_service.py`	certification spot-check
XSS token theft	Memory-only access token	`authStore.ts`	frontend audit
CSRF	JWT in header not cookie	API design	-
Rate abuse	SlowAPI 60/min	`main.py` middleware	-
Demo abuse	`DEMO_ENDPOINTS_ENABLED`	`routers/demo.py`	-
Malicious upload	File signature inspection	`evidence_service.py`	evidence tests
Host header attack	TrustedHostMiddleware	`main.py`, nginx config	DEPLOYMENT.md

MFA: DB columns exist - **NOT IMPLEMENTED** (no endpoints).

Source: `LBRO_PRODUCTION_CERTIFICATION.md`, `LBRO_Engineering_Audit.md`, `backend/tests/`

16. Security Incidents & Remediation History

All items from audit documents. Issues marked **Previously identified and remediated** where fixed in current code.

Issue	Danger	Detection	Fix	Status
Self-register as adm	Instant privilege es	Production audit P0	Default role `viewer	REMEDIATED
Broken login/refresh	Auth non-functional	P0 audit	Complete rewrite	REMEDIATED
Cross-project IDOR o	Data leakage	P1 audit	project_id filters i	REMEDIATED
Evidence IDOR	Download other tenan	P1 audit	JOIN through inciden	REMEDIATED
API keys plaintext i	DB dump exposes keys	Engineering audit	HMAC hash migration	REMEDIATED
bytes.lower() on evi	Content inspection b	P1 audit	Binary-safe comparis	REMEDIATED
Reports NameError	Reports crash	P0 audit	project_id param add	REMEDIATED
Global dashboard met	Cross-tenant stats l	Security review	Project-scoped queri	REMEDIATED
Demo endpoints expos	Unauthorized seeding	Config review	DEMO_ENDPOINTS_ENABL	REMEDIATED
ML scaler not loaded	Wrong predictions	P1 audit	Load scaler.pkl in _	REMEDIATED
TrustedHost / nginx	400 Invalid host hea	EC2 deployment	nginx proxy Host fix	REMEDIATED (local)

Lesson: Always scope queries by `project_id` at the service layer, not just the router.

Source: `LBRO_PRODUCTION_CERTIFICATION.md`, `LBRO_Engineering_Audit.md`, `LBRO_v1_Production_Audit_Report.md`, `docs/DEPLOYMENT.md`

17. DevOps & Docker

17.1 Docker Compose Services (Local)

Service	Image	Role
postgres	postgres:16-alpine	Database
localstack	localstack:3.4	S3/SQS/Secrets emulation
migrate	Dockerfile.api	Alembic one-shot
api	Dockerfile.api	FastAPI on :8000
worker	Dockerfile.worker	SQS consumer
frontend	frontend/Dockerfile	Nginx + React on :3000
seed	Dockerfile.api	Demo data one-shot

17.2 Production

`docker-compose.prod.yml` - EC2 deployment documented in `docs/DEPLOYMENT.md`. Non-root containers, multi-stage builds.

Why separate worker? Decouples long-running SQS poll + notification retry from API request threads.

Source: `docker-compose.yml`, `docker/Dockerfile.api`, `docker/Dockerfile.worker`

18. AWS & Terraform

18.1 Terraform-Defined Resources (IaC Template)

Canonical root: `terraform/main.tf` - modules: networking, sqs, s3, iam, ecs, rds, monitoring

AWS Service	LBRO Usage
ECS Fargate	API, worker, frontend services
RDS PostgreSQL	Managed database
SQS	Incidents + notifications queues + DLQ
S3	Evidence, reports, ML models buckets (Object Lock
Secrets Manager	App secrets, DB credentials
ALB	Load balancing
CloudWatch	Logs + alarms -> SNS
IAM	Task roles for S3/SQS/Secrets
VPC	Network isolation

UNVERIFIED as live production: Terraform defines target architecture; actual deployment may be EC2 + Docker Compose per `docs/DEPLOYMENT.md`.

Alternate stacks: `terraform/environments/dev|prod/` include WAF, EventBridge, backup modules not in canonical root.

Source: `terraform/`, `docs/DEPLOYMENT.md`

19. CI/CD

`.github/workflows/ci.yml`: On push/PR - ruff lint, pytest with coverage, Docker build, frontend lint/typecheck/build, terraform validate, Trivy filesystem scan, Bandit SAST.

`.github/workflows/deploy.yml`: On main/tags - build/push ECR, deploy ECS staging/production.

Source: `.github/workflows/`

20. Engineering Trade-Offs

Decision	LBRO Choice	Rationale	Cost
Event processing	Sync in API	Simpler demo path, immedi	API latency under load
Evidence storage	PostgreSQL BYTEA	Simplicity, transactional	DB size, backup cost
Auth	JWT + refresh	SPA-friendly	Revocation complexity (JT
API keys	HMAC not bcrypt	Fast prefix-index lookup	SECRET_KEY compromise = a

ML	GaussianNB pickle	Lightweight tabular	No online retraining
Multi-tenancy	Project table + members	Clear isolation boundary	No org-level hierarchy ye

21. Challenges Faced (From Audit History)

1. **Authentication broken at runtime** - truncated methods -> full rewrite + 654 tests 2. **Project isolation IDOR** - service-layer project_id JOINs + 11+ isolation tests 3. **API key plaintext** - migration 012 HMAC hashing 4. **Evidence content inspection** - bytes.lower() bug -> binary-safe checks 5. **Docker host header on EC2** - nginx Host header + ALLOWED_HOSTS config 6. **ML model not in repo** - heuristic fallback ensures uptime 7. **Compliance report scoping** - all sub-queries through project filter helper

22. Testing Strategy

Backend (654 tests, 37 files): Unit, integration, RBAC, project isolation, invitations, auth, evidence, ML, worker.

Frontend (50 tests, 5 files): Component and store tests via Vitest.

Security-focused tests: `test_project_isolation.py`, `test_project_invitations.py`, `test_api_keys.py`, `test_auth.py`

CI: SQLite in-memory for pytest speed - `backend/tests/conftest.py`

Historical vs current: Earlier audits cite ~428 tests; current verified count is **654**.

23. Performance & Scalability

Current bottlenecks:

- Synchronous ML in API event handler (CPU-bound in async loop)

- Evidence

24. Failure Scenarios

Failure	Behavior
PostgreSQL down	API 503, health check fails
SQS unavailable	Incident enqueue fails; events still sync-process
Worker crash	Messages return to queue after visibility timeout
ML model missing	Heuristic fallback, needs_review flagged
Invalid API key	401 Unauthorized
Expired JWT	401, frontend refresh attempt
Unauthorized project	403 Forbidden + audit log
Evidence upload fail	413/400 with validation error
AWS unavailable	Local docker still works; prod deploy fails

25. Presentation Guide (10-15 min)

Slide	Show	Say	Key Point
1 Problem	Breach timeline graphic	Teams juggle logs, eviden	LBRO unifies the workflow
2 Solution	LBRO dashboard	Multi-tenant incident pla	Not just a SIEM
3 Architecture	Diagram §2.1	React->FastAPI->PostgreSQ	Sync ingest, async worker
4 User flow	Register->project->API ke	Self-service tenant setup	Project isolation
5 Event ingest	curl/SDK example	API key resolves project,	Security by design
6 ML	ML Insights page	GaussianNB + heuristic fa	Honest about fallback
7 Incidents	Incident detail	Auto-create on high sever	Classification metadata
8 Evidence	Upload + verify	SHA-256 + chain of custod	Integrity provable
9 Compliance	Compliance dashboard	GDPR/HIPAA/DPDPA from rea	No fake percentages

10 RBAC	Members/roles UI	admin/analyst/viewer + su	IDOR tests prove isolatio
11 Deploy	Docker compose / EC2	Containers + optional AWS	IaC ready
12 Security	Audit summary	P0/P1 all remediated	654 tests
13 Demo	Live event -> incident	Send event via API key	End-to-end proof
14 Challenges	IDOR, API keys	Service-layer scoping les	Real engineering stories
15 Future	S3 evidence, MFA, email	Labeled NOT IMPLEMENTED	Honest roadmap

26. Live Demo Script

1. **Register** - "Public registration creates a viewer-role account; production disables this." 2. **Create project** - "Each project is an isolation boundary with its own API key." 3. **Generate API key** - "Shown once; stored as HMAC hash - migration 012 remediated plaintext storage." 4. **Send event** - `curl -H "Authorization: Bearer proj_..." -d '{"event_type":"port_scan","severity":"high",...}'` 5. **Show incident** - "High severity auto-created incident with ML or heuristic classification." 6. **Upload evidence** - "SHA-256 computed; chain of custody entry created." 7. **Compliance** - "Obligations generated from jurisdictions on the incident." 8. **Invite user** - "Admin sends invitation; email match required on accept." 9. **Switch project** - "User A cannot see User B's project data - isolation test proven."

27. Interview Questions & Answers (50+)

Architecture

Q1: What is LBRO?

- **Short:** Multi-tenant security incident response platform with ML classification, evidence vault, and compliance tracking.

- **Detail:
isolation.

- **Short:** Async Python, automatic OpenAPI, Pydantic validation.

- **Detail:*

- **Short:** ACID, relational integrity, BYTEA for current evidence storage.

- **Detail:*

- **Short:** Reproducible dev/prod parity across api/worker/frontend/postgres.

- **Detail:*

- **Short:** Managed async queue for incident re-classification and notifications.

- **Detail:*

Security & RBAC

Q6: How does RBAC work?

- **Short:** Platform super_admin + project admin/analyst/viewer with 32 permissions.

- **Detail:*

- **Short:** Every query scoped by accessible project_ids from membership + ownership.

- **Detail:*

- **Short:** DB compromise doesn't expose usable keys.

- **Detail:*

- **Short:** Stateless SPA auth with refresh token rotation.

- **Detail:*

- **Short:** Ignored for event ingest; JWT routes resolve scope server-side.

- **Detail:*

Database

Q11: What tables store evidence?

- **Short:** `evidence` (file_data BYTEA, sha256_hash) + `chain_of_custody`.

- **Detail:*

- **Short:** Multi-user, multi-project membership with per-project roles.

- **Detail:*

ML

Q13: Why GaussianNB?

- **Short:** Lightweight tabular classifier for CICIDS2017 features.

- **Detail:*

- **Short:** Heuristic rules + severity mapping; needs_review=True.

- **Detail:*

- **Short:** 0.75 in config.py (not 65% as README claims).

- **Detail:*

Cloud & DevOps

Q16: What AWS services does LBRO use?

- **Short:** ECS, RDS, SQS, S3, Secrets Manager, CloudWatch (Terraform template).

- **Detail:*

- **Short:** GitHub Actions: lint, 654 pytest, Trivy, Bandit, Docker build, terraform validate.

- **Detail:*

System Design

Q18: How would you scale to millions of events?

- **Short:** Queue-based ingest, worker pool, evidence to S3, read replicas.

- **Detail:*

- **Short:** Synchronous event processing + PostgreSQL evidence bytes.
- **Detail:*

- **Short:** S3 evidence, MFA, email invitations, async event queue, online ML retraining.
- **Detail:*

Q21: How does chain of custody work technically?
- **Short:** Every evidence access creates a ChainOfCustody row with action, user, hash_at_time, IP.
- **Detail:*

- **Short:** bcrypt via Passlib CryptContext.
- **Detail:*

- **Short:** Admin creates invite with email+role; token hashed; 7-day expiry; accept creates ProjectMember.
- **Detail:*

- **Short:** Platform role bypassing project restrictions for platform management.
- **Detail:*

- **Short:** `accessible_project_ids()` returns union; frontend ProjectSwitcher filters UI.
- **Detail:*

- **Short:** Worker polls SQS for incident ML re-classification and notification dispatch.
- **Detail:*

- **Short:** Attaches Bearer token; refreshes on 401 using sessionStorage refresh token.
- **Detail:*

- **Short:** POST /events/batch accepts up to 100 events per request.
- **Detail:*

- **Short:** Server-Sent Events for live event feed per project API key.
- **Detail:*

- **Short:** Worker retries up to 3 times via SQS; retry_count on notification model.
- **Detail:*

- **Short:** Aggregates project-scoped incidents, compliance, evidence counts.
- **Detail:*

- **Short:** POST /demo/generate seeds sample data when DEMO_ENDPOINTS_ENABLED=True.
- **Detail:*

- **Short:** Versioned, reversible schema changes across 14 migrations.
- **Detail:*

- **Short:** AsyncSession with asyncpg driver for non-blocking DB I/O.
- **Detail:*

- **Short:** ProtectedRoute + permission checks on admin pages.
- **Detail:*

- **Short:** POST regenerate-key generates new proj_* key; old hash replaced.
- **Detail:*

- **Short:** Per-user keys on users table; X-API-Key header auth alternative to JWT.
- **Detail:*

- **Short:** Notes belong to incident; incident belongs to project.
- **Detail:*

- **Short:** audit_logs table captures user actions with resource_type, path, details.
- **Detail:*

- **Short:** Only GET /health exists - NOT /health/ready.
- **Detail:*

- **Short:** Emulates S3, SQS, Secrets Manager in local docker-compose.
- **Detail:*

- **Short:** UNVERIFIED remote state config - local terraform/ directory is template.
- **Detail:*

- **Short:** Only if is_immutable=False; default is immutable.
- **Detail:*

- **Short:** Size limits, content inspection with binary-safe signature checks.
- **Detail:*

- **Short:** Pydantic validators on RegisterRequest - minimum length, complexity.

- **Detail:*

- **Short:** SlowAPI middleware - 60 requests/minute per IP default.

- **Detail:*

- **Short:** CORSMiddleware with CORS_ORIGINS from environment.

- **Detail:*

- **Short:** P0 auth handler truncation - login/refresh/logout broken at runtime.

- **Detail:*

- **Short:** Attempt IDOR via incident/evidence UUID - blocked by project_id JOIN.

- **Detail:*

- **Short:** Run pytest test_project_isolation.py and test_project_invitations.py.

- **Detail:*

28. Hard Interview Questions - Design Challenges

Q: Isn't this just a SIEM? A: LBRO focuses on incident response workflow (classify->incident->evidence->compliance deadline), not petabyte log storage or correlation rules at scale. It complements SIEM, not replaces Splunk/Elastic.

Q: Why not Kafka instead of SQS? A: Current scale and team ops favor managed SQS with DLQ. Kafka adds operational burden without current throughput need. Terraform could swap queue layer.

Q: Why PostgreSQL for evidence? A: Current implementation prioritizes simplicity and transactional integrity. README/Terraform mention S3 - runtime uses BYTEA. We'd migrate evidence to S3 before enterprise scale.

Q: Isn't storing files in PostgreSQL expensive? A: Yes at scale - acknowledged trade-off. BYTEA deferred loading mitigates read cost; S3 is documented future path.

Q: Why not deep learning? A: 77 numeric tabular features from CICIDS2017; GaussianNB/RF sufficient, interpretable, CPU-only. No labeled dataset at production volumes for neural net advantage.

Q: How do you prevent API key theft? A: HTTPS, hashed at rest, prefix-only index, rate limiting, project-scoped permissions. Leaked key only accesses one project - rotate via regenerate-key.

Q: Can a user access another project? A: Not if authorization helpers applied - proven by isolation tests. Privileged platform admin is intentional bypass, audit-logged.

Q: What happens if ML is down? A: Heuristic fallback - events still ingest, incidents still create, flagged needs_review.

Q: How do you know compliance percentages are real? A: `compliance_percentage()` returns None when zero obligations; otherwise met/total from persisted records only.

Q: What would you change before enterprise deployment? A: S3 evidence, MFA, email notifications, async event queue, secrets rotation, ECS/Fargate prod, WAF, penetration re-test, remove demo endpoints.

29. Elevator Pitches

30 Seconds

"LBRO is a multi-tenant security incident response platform. External apps send security events via project API keys; we classify them with ML, auto-create incidents, preserve evidence with SHA-256 chain of custody, and track GDPR, HIPAA, and DPDPA compliance deadlines - all with strict project isolation and role-based access."

1 Minute

"LBRO solves the gap between detecting a security event and meeting regulatory breach notification requirements. Built with FastAPI and React, it lets teams create isolated projects, connect applications through API keys, and ingest events that are classified by a GaussianNB model - with a heuristic fallback if the model isn't deployed. High-severity events become incidents automatically. Analysts upload evidence stored with integrity hashes and chain of custody, while the compliance engine generates real obligations from incident metadata. RBAC enforces admin, analyst, and viewer roles per project, and our test suite includes dedicated IDOR prevention tests. We remediated all P0/P1 security findings from our production audit, including API key hashing and cross-project access bugs."

3 Minutes

(Expand 1-minute version with architecture diagram walkthrough, sync vs async paths, PostgreSQL evidence honesty, Terraform AWS template vs EC2 deployment reality, 654 backend tests, invitation system, Super Admin platform layer, and v3 roadmap: S3 evidence, MFA, async ingest.)

30. Personal Contribution

UNVERIFIED from repository authorship alone.

Use this template in interviews:

- "I contributed to [specific module] - e.g., project invitations in `invitation_service.py` / frontend members UI."

- "I fixed [s

31. Glossary

Term	Definition
RBAC	Role-Based Access Control - permissions mapped to
IDOR	Insecure Direct Object Reference - accessing resou
JWT	JSON Web Token - signed claims for stateless auth
JTI	JWT ID - unique token identifier for revocation
HMAC	Hash-based Message Authentication Code - API key s
SHA-256	Cryptographic hash for evidence integrity
SQS	AWS Simple Queue Service - async job queue
RDS	AWS Relational Database Service - managed PostgreS
ECS/Fargate	AWS container orchestration - serverless container
ORM	Object-Relational Mapping - SQLAlchemy models
Alembic	Database migration tool for SQLAlchemy
Multi-tenancy	Multiple isolated customers/projects on one deploy
Chain of custody	Audit trail of evidence access and integrity snaps
SIEM	Security Information and Event Management - log ag
IOC	Indicator of Compromise - forensic artifact
BYTEA	PostgreSQL binary column type - current evidence s

32. Final Cheat Sheet

Architecture in One Line

React SPA + FastAPI + PostgreSQL + optional SQS worker + ML classifier (GaussianNB/heuristic).

Tech Stack

Python, FastAPI, React, TypeScript, PostgreSQL, SQLAlchemy, Alembic, Docker, AWS (Terraform template), scikit-learn, pytest, Vitest.

Security Essentials

- bcrypt passwords

- JWT + J

ML Essentials

- 77 features, 15 classes

- v2.0.0-nb

Evidence

- PostgreSQL file_data (NOT S3 runtime)

- SHA-256

Compliance

- GDPR 72h, HIPAA 60d, DPDPA 72h

- Real me

Key APIs

POST /events (API key), POST /auth/login (JWT), GET /projects, POST /projects/{id}/invitations

Test Counts

654 backend, 50 frontend (current verified)

Major Problems Solved

Admin self-registration, IDOR, plaintext API keys, broken auth handlers, evidence inspection bug

Documentation Mismatches to Mention Proactively

README: S3 evidence, API-key login, 78 features, 65% threshold, SQS event ingest - all differ from code

NOT IMPLEMENTED

MFA endpoints, email invitation sending (unless SMTP configured), S3 evidence at runtime, live ML retraining

10 Things You Must Remember

1. LBRO = incident response workflow, not SIEM 2. Events processed synchronously in API 3. project_id from API key ONLY for ingest 4. Evidence in PostgreSQL BYTEA today 5. ML fallback always available 6. 654 tests including IDOR suite 7. API keys HMAC-hashed (migration 012) 8. Compliance % from real records only 9. super_admin bypasses project scope (audited) 10. README ? implementation - cite code first

33. Source Reference Index

Topic	Primary Source
Event ingestion	`backend/app/routers/events.py`
Project access	`backend/app/core/project_access.py`
RBAC	`backend/app/core/rbac.py`
API keys	`backend/app/core/api_keys.py`
Auth	`backend/app/services/auth_service.py`
Evidence	`backend/app/services/evidence_service.py`
Compliance	`backend/app/services/compliance_service.py`
ML	`backend/app/ml/classifier.py`
Worker	`backend/app/workers/main.py`

Models	`backend/app/models/`
Migrations	`backend/app/migrations/versions/`
Frontend routes	`frontend/src/routes/AppRouter.tsx`
Docker	`docker-compose.yml`, `docker-compose.prod.yml`
Terraform	`terraform/main.tf`
CI	`.github/workflows/ci.yml`
Deployment	`docs/DEPLOYMENT.md`
Multi-project	`docs/MULTI_PROJECT.md`
Audits	`LBRO_PRODUCTION_CERTIFICATION.md`, `LBRO_Engineer

34. Appendix A - Complete Migration History

Rev	File	What Changed
001	001_initial_schema.py	users, incidents, evidence, notif
002	002_rbac_roles.py	Expanded role enum
003	003_simplify_roles.py	admin/analyst/viewer simplificati
004	004_sanitize_legacy_roles.py	Data cleanup for role migration
005	005_revoked_tokens.py	JWT JTI blacklist table
006	006_evidence_postgres_storage.py	evidence.file_data BYTEA column
007	007_projects.py	projects table, project_id on inc
008	008_compliance_persistence.py	compliance_obligations, complianc
009	009_add_missing_indexes.py	Performance indexes on FKs
010	010_platform_layer.py	project_members, security_events,
011	011_investigation_notes.py	investigation_notes table
012	012_hash_api_keys.py	HMAC hash for API keys at rest
013	013_backfill_project_members.py	Owner membership backfill
014	014_project_invitations.py	project_invitations table

Source: `backend/app/migrations/versions/`

35. Appendix B - Frontend Pages & Routes

Route	Page Component	Purpose
/login	LoginPage	Email/password authentication
/register	RegisterPage	User registration
/dashboard	DashboardPage	Project-scoped metrics summary
/incidents	IncidentsPage	Incident list and filters
/incidents/:id	IncidentDetailPage	Single incident with timeline, no
/incidents/new	CreateIncidentPage	Manual incident creation
/evidence	EvidencePage	Evidence list across incidents
/compliance	CompliancePage	Compliance dashboard
/compliance/audit	ComplianceAuditPage	Obligation audit view
/projects	ProjectsPage	Project list and creation
/projects/:id	ProjectOverviewPage	Single project dashboard
/projects/:id/settings	ProjectSettingsPage	Members, invitations, API key
/projects/:id/integrations	IntegrationsPage	SDK download, connection guide
/projects/:id/events	LiveEventsPage	Live event stream view
/ml	MLInsightsPage	ML model stats and classification
/notifications	NotificationsPage	Regulatory notification managemen
/users	UsersPage	User management (admin)
/audit	AuditLogsPage	Audit trail viewer
/platform	PlatformDashboard	Super Admin platform view

Source: `frontend/src/routes/AppRouter.tsx`

36. Appendix C - Sample Event Ingest Request

```
POST /api/v1/events HTTP/1.1
Host: ap
```

Response (201): SecurityEvent JSON with id, processing_status=processed, ml_attack_category, incident_id if auto-created.

Source: `backend/app/routers/events.py`, `SecurityEventCreate` schema

37. Appendix D - RBAC Permission Matrix (Summary)

Permission	Viewer	Analyst	Admin	Super Admin
incident:read	Yes	Yes	Yes	Yes
incident:create	No	Yes	Yes	Yes
incident:update	No	Yes	Yes	Yes
evidence:upload	No	Yes	Yes	Yes
evidence:download	Yes	Yes	Yes	Yes
report:generate	No	No	Yes	Yes
user:manage	No	No	Yes	Yes
apikey:rotate	No	No	Yes	Yes
platform:*	No	No	No	Yes

Source: `backend/app/core/rbac.py::ROLE_PERMISSIONS`

38. Appendix E - Documentation vs Implementation Conflicts (Complete List)

Topic	README/Doc Claim	Actual Implementation	Label
Login method	API key exchange	Email/password JWT	MISMATCH
API key hash	bcrypt	HMAC-SHA256	MISMATCH
JWT expiry	1 hour	30 minutes	MISMATCH
Rate limit	100/min	60/min	MISMATCH
Evidence storage	S3 Object Lock	PostgreSQL BYTEA	MISMATCH
ML features	78	77	MISMATCH
ML threshold	65%	75% (0.75)	MISMATCH
Event pipeline	SQS async	Sync in API	MISMATCH
Readiness probe	/health/ready	/health only	MISMATCH
Frontend auth	X-API-Key	Bearer JWT	MISMATCH
MFA	Mentioned	DB columns only, no endpo	NOT IMPLEMENTED
Email invitations	Implied	SMTP required, else no se	NOT IMPLEMENTED
ECS production	Architecture diagram	EC2 Docker documented	UNVERIFIED
ML model in repo	Implied trained model	registry.json only, no .p	UNVERIFIED deploy

Interview tip: Proactively cite these - demonstrates codebase-first honesty.

39. Appendix F - Red-Team Test Scenarios

1. **Cross-project incident read:** User A requests User B incident UUID -> expect 403/404 2. **Evidence download IDOR:** Manipulate evidence ID from another project -> blocked by JOIN 3. **project_id in event body:** Send different project_id -> ignored; key determines project 4. **Self-register as admin:** Fixed - always viewer role 5. **Expired JWT:** 401 after expiry; refresh if valid refresh token 6. **Revoked JWT after logout:** JTI in revoked_tokens -> 401 7. **Demo endpoint in prod:** DEMO_ENDPOINTS_ENABLED=False -> 404 8. **Compliance mark_met cross-project:** Requires project ownership -> 403 9. **Global dashboard without project scope:** Fixed - project_id required 10. **API key prefix brute force:** Prefix index only narrows candidates; HMAC verify required

Source: `backend/tests/test_project_isolation.py`, `LBRO_PRODUCTION_CERTIFICATION.md`

End of LBRO Interview & Architecture Master Booklet